

THE HONG KONG UNIVERSITY OF SCIENCE & TECHNOLOGY
Department of Computer Science and Engineering
ARIN5103: Foundations of Machine Learning / MAIE5212: Machine Learning
Fall 2025 Assignment 1
Due time and date: 11:59pm, Oct 28 (Tue), 2025.

IMPORTANT NOTES

- **Your grade will be based on the correctness and clarity.**
- **Late submission: 25 marks will be deducted for every 24 hours after the deadline.**
- **ZERO-Tolerance on Plagiarism: All involved parties will get zero mark.**

Q1: Representation using perceptrons

Use *ONE* perceptron to represent the boolean expression:

$$\neg a \vee b \vee \neg c,$$

where a, b, c are Boolean variables (with value 0 or 1). You have to specify the weight, bias term and activation function in the perceptron.

Q2: Computation graph

Draw the computation graph for the function

$$f(x, y, z, w) = \text{sigmoid}(\max(x, y) + z \cdot w),$$

where $\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$. Note that you should not directly use sigmoid as a function in the computation graph. With $x = 1, y = -2, z = 0.5$ and $w = 2$, obtain the output and all the gradients in the computation.

Q3: Empirical comparison of MLP and CNN on image classification

In this question, you will empirically compare the Convolutional Neural Networks (CNNs) and Multilayer Perceptrons (MLPs) for image classification. You will implement, train, and evaluate both architectures on the CIFAR-10 dataset, followed by a detailed analysis of their performance and complexity.

Part A: Data Loading and Preparation

Prepare the CIFAR-10 dataset for training and testing by following these steps.

1. **Load Dataset:** Use `torchvision.datasets.CIFAR10` to load the training and testing sets. Randomly split the test set into two equal halves: one for validation and one for testing.
2. **Data Transformation:** Define a transformation pipeline that converts images to PyTorch tensors and then normalizes them using a mean of `[0.4914, 0.4822, 0.4465]` and a standard deviation of `[0.2470, 0.2435, 0.2616]`.
3. **DataLoaders:** Create `DataLoader` for the training, validation and testing sets, each with a batch size of 64.

Part B: MLP Implementation and Training

First, you will build and train a standard MLP model.

Model Architecture. Implement an MLP that takes a flattened CIFAR-10 image ($3 \times 32 \times 32 = 3072$ features) as input. The network should have the following structure:

- a) First, a fully connected layer with 1024 output units, followed by a ReLU activation and a dropout layer ($p = 0.5$).
- b) Next, a fully connected layer with 512 output units, followed by a ReLU activation and a dropout layer ($p = 0.5$).
- c) A fully connected layer mapping to the 10 CIFAR-10 classes.

Training and Evaluation. Train the model for **10 epochs** using the Adam optimizer (`torch.optim.Adam`) with a learning rate of 0.001 and weight decay of 0.0001 and the Cross-Entropy Loss function (`nn.CrossEntropyLoss`). During training, record the training loss, validation loss, training accuracy, and validation accuracy for each epoch.

Part C: CNN Implementation and Training

Next, you will build and train a simple CNN model.

Model Architecture. Implement a CNN that takes the original $3 \times 32 \times 32$ image tensor as input. The network should have the following architecture:

[Block 1]

- A convolution layer: 3 input channels, 32 output channels, kernel size 3×3 , zero-padding 1.
- ReLU activation
- A max-pooling layer: kernel size 3×3 , stride 2.

[Block 2]

- A convolution layer: 32 input channels, 64 output channels, kernel size 3×3 , zero-padding 1.
- ReLU activation
- A max-pooling layer: kernel size 2×2 , stride 2.

[Classifier Head]

- Flatten the output from block 2.
- A fully-connected layer projecting to 512 features, followed by a ReLU activation and a dropout layer ($p = 0.5$).
- A fully-connected layer mapping to the 10 classes.

Training and Evaluation. Train the CNN using the identical setup as the MLP. Record the same set of metrics (training/validation loss and accuracy) for each epoch.

Part D: Analysis and Discussion

Provide performance comparison of the two models.

1. Performance Comparison:

- Generate a single plot displaying the **validation accuracy** curves for both the MLP and the CNN over the 10 epochs. The y-axis should be "Validation Accuracy" and the x-axis "Epoch".
- Generate a second plot for the **validation loss** curves, formatted similarly.
- Report the final **test accuracies** of both models in a clearly formatted table.

2. Model Complexity:

- Implement a Python function that counts and returns the total number of trainable parameters for a given PyTorch model.
- Use this function to obtain the parameter counts for your MLP and CNN. Present the results in a table.

3. Discussion:

Discuss *why* the CNN outperformed the MLP in image classification.

Submission Guidelines

Please submit a pdf (A1.pdf) containing your answers to Q1 and Q2 and a Python notebook (A1.ipynb) containing code, running outputs and discussions for Q3. Please submit the assignment by uploading the files to Canvas. Note that the assignment should be clearly legible, otherwise you may lose some points if the assignment is difficult to read. Plagiarism will lead to zero point on this assignment.